ИИ

Общий инструмент на state machine как в Unreal и стратегический слой поверх него.

> Статус: предложение для обсуждения. Ни один новый класс из этого документа пока не существует. Примеры кода проверены компилятором, но реализации нет.

Коротко

1. **Основа — общий инструмент по образцу Unreal.** Иерархическая state machine как StateTree, контроллер, память, восприятие, поиск точек, навигация и отладка. Он подходит для любого жанра: стражник, турель, босс, сценарий миссии.
2. **Стратегия — слой поверх него.** Отряд является обычным контроллером с теми же автоматами и задачами. Добавляются только поле направлений, слоты и лёгкие бойцы-экземпляры.
3. **ИИ работает из коробки.** Подсистема встроена в движок, игра ничего не объявляет: наследуется от `AiCharacter` и описывает автомат.
4. **Сначала C++, потом мышь.** Автомат собирается кодом. На отдельном этапе появится файл `.fsm` и редактор графа, и оба способа дают один и тот же объект, поэтому игровой код не переписывается.
5. **Механизмы лежат в Engine, правила — в игре.** Движок не знает, что такое враг или мораль.

Что изменилось по сравнению с первой версией

- Основой стал общий инструмент. Плоский автомат заменён иерархическим, добавлены задачи, условия, службы, события, контроллер, восприятие и поиск точек.
- `AiBrain` убран: его место занял `Ai::Controller`.
- **Исправлена ошибка первой версии.** Подсистема `BattleAi` с `ENGINE_SUBSYSTEM()` в `Battle` не заработала бы: проект не подключён к генератору. Подсистема теперь встроена в `GameSession`.
- По умолчанию автомат думает каждый кадр, как в Unreal. Реже — по настройке, отряды — 5 раз в секунду.

Что повторяем из Unreal

StateTree выбран образцом потому, что это и есть иерархическая state machine: она соединяет состояния и переходы с выбором как в Behavior Tree.

Unreal	У нас	Отличия
AIController	`Ai::Controller`	Обычный объект C++, как `CharacterController`
StateTree: состояния, Enter Conditions, задачи, переходы	`Ai::StateMachine`	Те же правила выбора и порядка проверки
Linked state, Subtree	`Linked(machine)`	Вставка другого автомата
Evaluators, Global Tasks, BT Services	`Ai::Service`	Одно понятие вместо трёх
Blackboard	`Ai::Blackboard`	Типизированные ключи проверяет компилятор
Property binding	Ключи памяти в параметрах задач	Проще, без привязки свойств
AI Perception: зрение, слух, урон	`Ai::Perception`	Настройка в коде или свойствах
Generic Team Agent	`Ai::Team`, `Ai::Attitude`	
EQS: генераторы и тесты	`Ai::Query`	Основные генераторы и тесты

Unreal	У нас	Отличия
NavMesh	`NavGrid`	**Плоская сетка: один уровень, без этажей**
NavMeshBounds, NavModifier	`NavVolume`	Компонент сцены с тремя режимами
PathFollowing, MoveTo	`Controller::MoveTo`, `Ai::Tasks::MoveTo`	
Crowd, RVO	Разведение через `SpatialHash`	Проще, ORCA при необходимости
Smart Objects	`Ai::SmartObject`	Поздний этап
Gameplay Debugger, Visual Logger	`Ai::Debugger`, журнал переходов	Запись в файл не планируется
Mass	`Ai::Group`, поле направлений	Только для отрядов
Редактор StateTree	Файл `.fsm` и граф в редакторе	Отдельный этап
Behavior Tree	Отдельной системы нет	Выбор детей в автомате закрывает эту задачу
Blueprint	**Нет**	Только C++, вместо этого есть Hot Reload
NavLink, прыжки, несколько этажей	**Нет**	Нужна другая навигация, см. вопрос 5

Схемы

Уже есть в движке Новое: общий ИИ Новое: стратегия Код игры

Схема 1. Слои

Общий инструмент подходит для любого жанра. Стратегия только дополняет его. Движок не знает о врагах и морали.

Game/<Name> — правила конкретной игры

- **Юниты** — Guard : AiCharacter, Turret : CubeComponent. Появляются в палитре редактора
- **Автоматы** — Какие состояния есть и как они сменяются
- **Свои задачи и условия** — Attack, IsWounded: всё, что относится к правилам игры

↓ использует

Engine/AI — стратегия: отряды и толпа

- **Ai::Group** — Контроллер отряда: те же автоматы и задачи
- **FlowField** — Одно поле направлений на цель, кэш, расчёт порциями
- **Слоты и бойцы** — Лёгкие записи без контроллера, тело — MeshInstance

↓ построена на

Engine/AI — общий инструмент

- **Ai::StateMachine** — Иерархический автомат, один на тип юнита. StateTree
- **Ai::Controller** — Мозг: запускает автомат, держит память и чувства. AIController
- **Ai::Blackboard** — Память юнита с типизированными ключами. Blackboard
- **Ai::Perception** — Зрение, слух, урон, команды. AI Perception
- **NavGrid, A*, MoveTo** — Навигация и движение по пути. NavMesh, PathFollowing
- **Ai::Query, Debugger** — Поиск лучшей точки, отладка поведения. EQS, Gameplay Debugger

↓ опирается на

Engine — то, что уже работает

- **Character** — Move, базовый класс юнита
- **InstancedMesh Component** — Массовые юниты
- **GameSession** — Тик подсистем, Pause
- **Scene** — ComponentId, удаление
- **SaveArchive** — Сохранение, ссылки

Схема 2. Что внутри ИИ юнита

Как в Unreal: мозг отделён от тела. Один и тот же контроллер управляет персонажем, турелью, отрядом или работает вообще без объекта сцены.

Мир
Цели, шум, урон, события других юнитов

↓ что заметили

Ai::Controller — мозг одного юнита

- **Восприятие** — Видит цели, шлёт события
- **Память** — Target, LastSeen, Health
- **Автомат** — Выбирает состояние по условиям и событиям
- **Задачи** — Выполняют активные состояния
- **Движение** — Ищет путь, обходит соседей

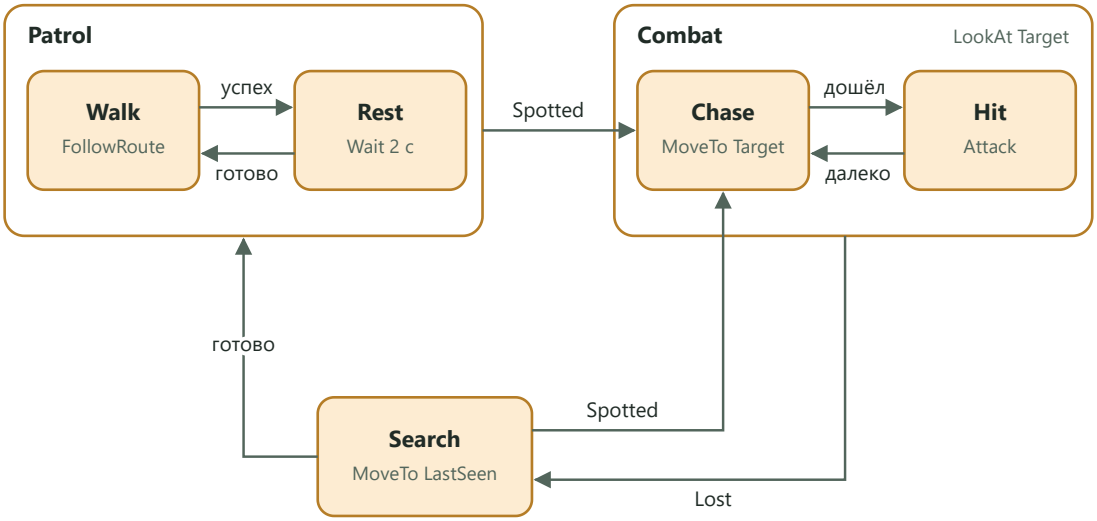
↓ куда идти и куда смотреть

- **Character** — Ходит через Move
- **Любой компонент** — Турель: видит и думает, но не ходит
- **Отряд** — Бойцы через SetInstancePosition
- **Нет тела** — Сценарий боя, фазы босса

Схема 3. Иерархический автомат

Стражник из руководства. Родитель сам не выполняется и входит в первого подходящего ребёнка, его задачи работают, пока активен любой ребёнок. Старт: Patrol, затем Walk.

Автомат Guard
Рамка — родитель, блок — состояние, внизу блока — его задача



Жизнь задачи
Все задачи состояния работают одновременно, первая завершившаяся завершает состояние

Enter — Вход в состояние → **Tick** — Пока статус Running → **Succeeded, Failed** — Или прервал переход → **Exit** — Вызывается всегда → **Переход** — От листа к корню

Схема 4. Как написать ИИ на C++

Четыре шага в коде. Подсистему объявлять не нужно, регистрировать классы вручную тоже.

1 Память

Ключи: имя и тип значения

```
Ai::Key<Ai::ObjectRef>
Target{"Target"};
```

2 Задачи и условия

Только то, что относится к правилам игры

```
class Attack : public
Ai::Task<AttackData>
```

3 Автомат

Состояния, задачи и переходы

```
chase.GoTo("Hit").OnSucceeded();
```

4 Юнит

Настройка чувств и запуск автомата

```
class Guard final : public
AiCharacter
```

5 Сборка

Генератор сам добавит класс в палитру, а поля — в инспектор

```
ENGINE_COMPONENT("Стражник")
```

6 Play и отладка

Состояние над юнитом и журнал переходов

```
ai.ActiveState(); ai.History();
```

Схема 5. Один кадр

ИИ встаёт в существующий шаг `Session->Tick`, цикл кадра не меняется. В Pause и в Edit он не работает.

`PreUpdate` `ActionInput.Tick` **`Session->Tick`** `Controller->Tick` `UpdateCamera` `TickComponents` `RenderFrame`

Внутри `Session->Tick` встроенная подсистема вызывает `Ai::World::Tick(dt)`:

1 Снять позиции

Убрать контроллеры удалённых объектов

2 Обновить соседей

SpatialHash пересобирается один раз

3 Восприятие

Проверки видимости по кругу, с бюджетом

4 Подумать

Службы, задачи, переходы

5 Поиск в рамках бюджета

A*, поля и запросы, не больше N за кадр

6 Желаемая скорость

Из пути, из поля или к своему слоту

7 Избегание

Разведение с соседями, запрет занятых клеток

8 Записать

Character::Move или SetInstancePosition

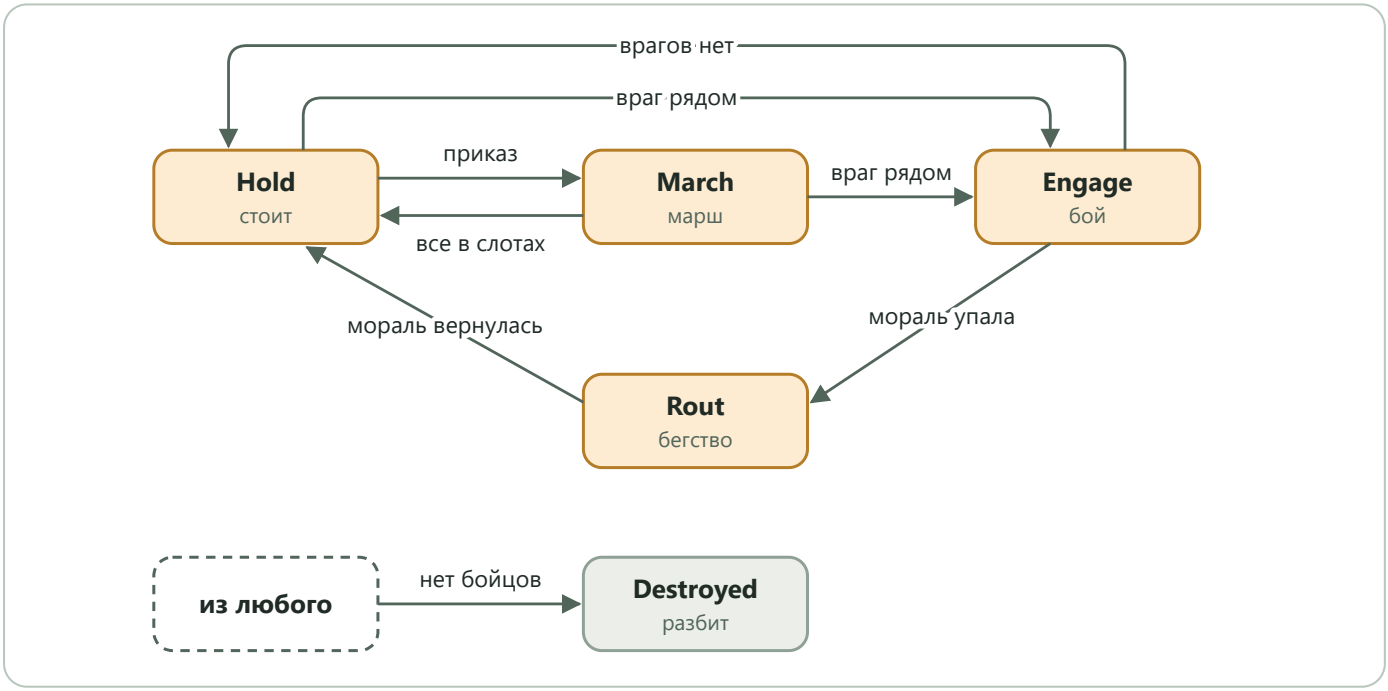
Схема 6. Как приказ становится движением

Различаются только источник направления и способ записи. Автоматы, задачи и избегание общие.

	Одиночный юнит (Character)	Толпа: отряд из MeshInstance
Кто решает	Задача автомата: `Ai::Tasks::MoveTo`	
Приказ	`Controller::MoveTo(point)`	`Group::MoveTo(point)`
Поиск	A* по NavGrid, запрос встаёт в очередь	Волна от цели, одно поле на цель и класс размера
Результат	Ломаная, спрямлённая по прямой видимости	Направление в каждой клетке; боец читает его за O(1)
Следование	К очередной точке. Нет продвижения — новый запрос	По полю, пока свой слот не станет виден напрямую
Избегание	Разведение через SpatialHash, запрет занятых клеток через NavGrid	
Запись	`Character::Move(direction, dt)` и поворот	`SetInstancePosition`, `SetInstanceYaw`
Ответ автомату	Статус: Waiting, Moving, Arrived, Unreachable, Stuck	
Цена растёт с	числом юнитов: один поиск на каждого	числом разных целей: от числа бойцов почти не зависит

Схема 7. Автомат отряда

Пример игрового кода для стратегии. Механизм тот же, что у стражника, но автомат один на сотню бойцов.



Архитектура

Что есть в движке и что из этого следует

Факт	Где	Следствие для ИИ
Класса `Unit` нет ни в `main`, ни в других ветках. Базовый класс — `Character`, в комментарии прямо сказано про ИИ	Character.h	`AiCharacter` наследуется от него. См. вопрос 2
`Move` только сдвигает позицию: без столкновений и без поворота	Character.cpp	Обход препятствий и поворот делает ИИ
Длина `direction` до 1 работает как газ	там же	Результат избегания идёт в `Move` без новых методов
`CharacterController` один на `Game` и привязан к вводу и камере	Game.h	Нужен свой контроллер, тот же приём с `Possess` по ID
Массовый юнит — `MeshInstance` в `InstancedMeshComponent`, в `Battle` их 30000	UNIT-RENDERING.md	Боец адресуется парой «компонент + индекс»
Удалить экземпляр нельзя, есть только `SetInstanceVisible`	InstancedMeshComponent.h	Индексы стабильны. Смерть бойца — скрыть и убрать агента
`Scene::Tick` обходит компоненты по имени	Scene.cpp	ИИ работает отдельным шагом, а не в `Tick` компонентов
`GameSubsystem::Tick` идёт после ввода, до контроллера и компонентов, не работает в Pause и Edit	GAME-SUBSYSTEMS.md	Готовое место для ИИ
`OnSceneReady` вызывается раньше `BeginPlay`	там же	Юнит может зарегистрироваться в `BeginPlay`
`Battle` не подключён к `ComponentGeneration.targets`	Battle.vcxproj	Игровая подсистема там не обнаружится, поэтому ИИ встроен в сессию
`ENGINE_COMPONENT` обнаруживает наследников, свойства генерирует `ENGINE_PROPERTY`	COMPONENT-GENERATION.md	Юниты попадают в палитру редактора без новой работы
`ENGINE_PROPERTY` работает только в `ENGINE_COMPONENT` и не умеет массивы	PROPERTY-MACROS.md	Для задач в редакторе генератор нужно расширить
В редакторе уже есть нодовый граф материалов	`Editor/Source/MaterialGraphWidget.cpp`	Опыт для редактора автоматов, но граф материалов — поток данных, а не состояния
`Scene::Raycast` обходит все компоненты, а в `InstancedMeshComponent` все экземпляры	Scene.cpp	Частые проверки видимости идут по сетке
Рельефа нет, в `Battle` земля плоская	BattleGame.cpp	Хватает 2D-сетки
`dt` умножен на `SimulationTimeScale`: 7.8 по умолчанию, 1.0 в проектах и редакторе	Game.h	См. раздел «Время»

Слои

Слой	Что входит	Кто пишет
Игра, `Game/<Name>/Source`	Юниты, свои задачи и условия, автоматы, правила	Автор игры
Стратегия, `Engine/AI`	`Ai::Group`, `FlowField`, слоты, бойцы-экземпляры	Движок
Общий ИИ, `Engine/AI`	`StateMachine`, `Controller`, `Blackboard`, `Perception`, `Query`, `NavGrid`, `Debugger`, стандартные задачи	Движок
Движок	`Character`, `Scene`, `GameSession`, `SaveArchive`, `InstancedMeshComponent`	Уже есть

Стратегия зависит от общего слоя, но не наоборот. Игра без отрядов просто не использует `Ai::Group`.

Владелец всего — `Ai::World` внутри встроенной подсистемы. Новый тип добавляется в `GameSession::Initialize`, поэтому ИИ есть в каждой игре, в том числе без генератора. Пока нет ни одного контроллера, тик выключен и ничего не стоит.

Иерархический автомат

Поведение описано подробно в руководстве. Здесь только решения.

- **Определение и исполнение разделены.** `Ai::StateMachine` неизменяем и общий для всех юнитов этого типа. У контроллера лежат только активная цепочка состояний и данные задач. Так устроен и StateTree: это экономит память и не даёт юнитам влиять друг на друга.
- **Методы задач** `const`**.** Данные юнита передаются отдельным параметром `Data&`, поэтому компилятор не даст записать их в общий объект. Проверено: такой код не собирается.
- **Память для** `Data` **выделяется один раз на контроллер.** Смещения считаются при сборке автомата, вход в состояние не вызывает аллокаций.
- **Правила выбора взяты из StateTree.** Родитель сам не выполняется и входит в первого ребёнка, разрешившего вход. Переходы проверяются от листа к корню. Задачи состояния работают одновременно, первая завершившаяся завершает состояние.
- **Смена состояния только через переходы.** Задача не может переключить автомат напрямую, поэтому логика видна в одном месте.
- **Автомат не требует тела.** Он работает на персонаже, турели, отряде и без объекта сцены.

Контроллер и тело

`Ai::Controller` — это мозг. Он запускает автомат, хранит память и восприятие, принимает `MoveTo`. Тело — то, чем он управляет.

Тело	Как создаётся	Ходит
`AiCharacter`	Наследник `Character`, контроллер создаётся в `BeginPlay`	Да, через `Character::Move`
Любой `GameComponent`	`world.Spawn(*this)`	Нет: статус `Unavailable`
Нет тела	`world.SpawnDirector()`	Нет
Отряд	`world.CreateGroup()`	Да, бойцы через `SetInstancePosition`

`AiCharacter` регистрируется в каталоге движка рядом с `Gameplay.Character`. Контроллер хранит ID тела, а не указатель, как и `CharacterController`. Персонаж игрока становится целью через `AddTarget`: его видят и обходят, но ИИ им не двигает.

Память, восприятие, поиск точек

Память. Значения по имени и типу. `Ai::ObjectRef` хранит ID объекта, поэтому при загрузке его перевязывает `SaveArchive::Reference`.

Восприятие. Кандидаты берутся из `SpatialHash`, поэтому юнит проверяет только соседей. Проверки видимости идут по кругу, не больше N за кадр. По умолчанию видимость считается по сетке, для точных сцен можно включить `Scene::Raycast`. Результат попадает в память и приходит событием.

Поиск точек. `Ai::Query` создаёт кандидатов, отсеивает и оценивает. Точек за кадр не больше N, результат приходит в задачу `RunQuery`.

Smart Objects. Объект сцены со слотами и резервированием, чтобы два юнита не сели на одну лавку. Это поздний этап, потому что сейчас нет сценария, где они нужны.

Навигация

NavGrid. Плоская сетка в XZ с размером клетки на сцену. В клетке два байта: стоимость прохода (255 — нельзя) и расстояние до препятствия, чтобы широкий юнит не лез в щель. Сетку задают компоненты `NavVolume` или код. Изменение в игре повышает `version` сетки, пути перепроверяются.

Одиночный юнит: A*.

1. `MoveTo` ставит запрос в очередь. За кадр обрабатывается не больше N узлов, поиск может занять несколько кадров.
2. Поиск идёт по 8 соседям без срезания углов.
3. Если цель недостижима, юнит идёт к ближайшей достижимой клетке и получает статус `Unreachable`.
4. Ломаная спрямляется по прямой видимости.
5. Нет продвижения — новый запрос. Несколько неудач подряд дают `Stuck`.

Состояния видят только `MoveTo` и статус. Если позже понадобится navmesh, заменится внутренность `Ai::World`, а не игровой код.

Стратегический слой

Отряд. `Ai::Group` наследуется от `Ai::Controller` и переопределяет `MoveTo`. Стандартная задача `Ai::Tasks::MoveTo` работает с ним без изменений. Автомат один на отряд, поэтому 30000 бойцов в отрядах по 100 дают 300 автоматов. Боец — запись в массиве с позицией, скоростью и радиусом, без контроллера.

Поле направлений (flow field). Считается волной от цели и хранит в каждой клетке направление к ней. Боец читает его за O(1).

1. `MoveTo` рассчитывает слоты на месте назначения. Занятые слоты сдвигаются на ближайшую проходимую клетку.
2. Поле берётся из кэша по ключу «цель, класс размера, версия сетки» или считается порциями.
3. Боец интерполирует направление между клетками, иначе будет дёргаться на границах.
4. Как только свой слот виден напрямую и близко, боец уходит с поля к слоту. Без слотов все рвались бы в одну клетку.
5. Дошедший боец засыпает, но остаётся препятствием для соседей.

Слабое место: цена растёт с числом разных целей. Одно поле на сетке `Battle` — это 484 тысячи клеток. 300 отрядов с 300 разными целями дают 145 миллионов клеток, то есть около 24 секунд при бюджете 100 тысяч клеток за кадр и 60 FPS. Что с этим делаем:

- один приказ нескольким отрядам — одно поле, слоты разводят их по местам;
- цель в ключе кэша округляется до крупной клетки, например 16 м;
- если замер покажет, что этого мало, поле считается только в окне вокруг отряда и цели.

Избегание соседей общее для всех тел. Юнит смотрит не больше 8 ближайших соседей через `SpatialHash` и отталкивается от них. В занятую клетку войти нельзя: юнит скользит вдоль стены или стоит. Смещение за кадр не больше размера клетки. Встречные колонны могут застревать, тогда следующий шаг — ORCA.

Порядок в кадре

`Ai::World::Tick` вызывается из встроенной подсистемы внутри `Session->Tick`. Цикл кадра не меняется.

1. Снять позиции и убрать контроллеры удалённых объектов.
2. Пересобрать `SpatialHash`.
3. Обновить восприятие в рамках бюджета.
4. Подумать: службы, задачи, переходы.
5. Продолжить A*, поля и запросы в рамках бюджета.
6. Посчитать желаемую скорость.
7. Применить избегание.
8. Записать результат в тела.

Всё идёт в игровом потоке: подсистемы привязаны к потоку владельца. Исключение из задачи идёт существующим путём: редактор уходит в Pause, runtime сообщает об ошибке кадра.

Время

`dt` — это реальное время, умноженное на `SimulationTimeScale`. Значение 1.0 ставят проекты и редактор, а `Battle` запускается своим `Main.cpp` и остаётся на 7.8. Там одна «секунда» ИИ равна примерно 0.13 реальной, а скорость 9 м/с на деле становится 70 м/с.

Решение: все таймеры ИИ считаются только по `dt`, а режим `Battle` с ИИ ставит `SetSimulationTimeScale(1.0f)`. Ограничение смещения за кадр защищает от большого шага и при другом масштабе.

Жизнь объектов и сохранение

- `OnSceneClosing` очищает контроллеры. Подсистема переживает смену сцены.
- Удалённое тело обнаруживается на первом шаге кадра, контроллер убирается сам.
- Как и у других подсистем, hot reload состояние не переносит. Контроллеры уничтожаются до выгрузки DLL.

Сохраняем: активные состояния по именам, память, приказы, восприятие, состав отрядов, свои поля контроллера. **Не сохраняем:** `Data` задач, пути, поля, `SpatialHash`, `NavGrid`.

После загрузки автомат возвращается в те же состояния и заново вызывает `Enter`, а сетка и пути строятся заново в `OnSaveGameLoaded`. Это простое и понятное правило: `Wait` начнёт отсчёт сначала, зато задачам не нужен код сохранения. Позиции уже сохраняет движок, у `InstancedMeshComponent` при этом лимит 100000 экземпляров на компонент.

Редактор автоматов

Это отдельный этап. Он не мешает начать с кода, потому что `MachineBuilder` и файл дают один объект.

```
{
  "version": 1,
  "name": "Guard",
  "states": [
```

```
    {"name": "Patrol", "children": [
      {"name": "Walk",
       "tasks": [{"type": "Ai.FollowRoute"}],
       "transitions": [{"to": "Rest", "on": "succeeded"}]}
    ]}
  ]
}
```

Что потребуется:

- **Файл `.fsm`:** JSON с версией, как `.mat`, загрузка и проверка ошибок, распознавание в браузере файлов.
- **Каталог задач:** `ENGINE_AI_TASK` и `ENGINE_AI_CONDITION` в `ComponentGenerator`, поддержка `ENGINE_PROPERTY` вне компонентов.
- **Окно редактора:** дерево состояний и граф переходов. Граф материалов — поток данных, поэтому основу виджета придётся дорабатывать.
- **Отладка в Play:** подсветка активного состояния через Edit session API.

Это самый дорогой этап, поэтому его место в очереди — вопрос 1.

Оценки

Память посчитана для `Battle`: 174 юнита в ряду × 8 м дают поле 1392 м. При клетке 2 м это 696 × 696 = 484 416 клеток. **Время ниже — цели, а не замеры.**

Что	Значение
`NavGrid`, 2 байта на клетку	≈ 0.9 МБ
Готовое поле, 1 байт на клетку	≈ 0.5 МБ
Бойцы, около 64 байт на каждого	≈ 1.9 МБ на 30000
Обдумываний за кадр: 300 отрядов, 5 Гц, 60 FPS	25
Цель для всего `Ai::World::Tick`	не больше 4 мс при 30000 бойцов, уточнить по замеру `Battle`

Почему не иначе

Вариант	Почему не сейчас
Отдельный Behavior Tree	Две системы вместо одной. Иерархический автомат закрывает выбор поведения, и вы выбрали state machine
Плоский автомат	Переходы размножаются: «умер» пришлось бы писать из каждого состояния
Задачи с обычными полями и копией на юнита	Проще писать, но память растёт с числом юнитов, а ошибка с общими данными не ловится
Navmesh (Recast/Detour)	Сильнее на сложной геометрии, но рельефа нет, это новая зависимость, а толпе всё равно нужна сетка
A* на каждого бойца	30000 поисков на один приказ
Логика в `Character::Tick`	Порядок зависит от имён, нет общего бюджета, толпа не покрыта
Каждый боец — `Character`	Объект сцены со своей отрисовкой и обновлением, на тысячах не работает
ECS	Правила проекта просят его избегать, плотного массива внутри `Ai::World` достаточно
Потоки в первой версии	Сначала замер. Если упрёмся, распараллеливается шаг избегания

Что поменяется в существующем коде

- `GameSession::Initialize`: встроенная подсистема ИИ.
- `Engine.vcxproj`, `StandardTypes.h`: новые файлы, типы `Gameplay.AiCharacter` и `Gameplay.NavVolume`.
- `BattleGame`: режим с ИИ ставит масштаб времени 1.0 и выключает `UpdateUnits`.
- `Game.cpp`: отладочные линии. Оверлей рисуется только при `bEmbedded`, поэтому в Play редактора он есть, а в самостоятельном `Battle` нет.
- На этапе редактора: `ComponentGenerator` и `Editor`.
- `Character` менять не нужно.

Этапы

Одна задача за раз, после каждой отчёт. Этапы 1–4 дают общий рабочий ИИ, этапы 8–10 — стратегию.

Этап	Что делаем	Как проверяем
1. Автомат	`StateMachine`, `Task`, `Service`, `Blackboard`, `MachineBuilder`	Тесты без сцены: порядок вызовов, выбор детей, переходы от листа к корню, события, `Linked`, ошибки в именах
2. Навигация	`NavGrid`, `NavVolume`, A*, спрямление	Карты в тестах: обход стены, нет пути, широкий юнит, углы
3. Контроллер	`Ai::World`, встроенная подсистема, `Controller`, `AiCharacter`, `MoveTo`, стандартные задачи, журнал	Безоконный прогон: юнит дошёл и не был в занятой клетке. Запуск сцены
4. Восприятие	`SpatialHash`, зрение, слух, урон, команды	Стражник из руководства работает целиком
5. Сохранение	Запись контроллеров и памяти	Сохранить в бою, загрузить: то же состояние и цель
6. Отладчик	Состояния над юнитами, память, путь, конус зрения	Разбор поведения стражника без отладчика Visual Studio
7. Поиск точек	`Ai::Query`, генераторы, тесты	Юнит выбирает укрытие, которое не видит враг
8. Избегание	Разведение, запрет входа	Нет наложений и дрожания, встречные колонны
9. Толпа	`Ai::Group`, `FlowField`, слоты, бойцы, приказы в `Battle`	1000 бойцов: поле построено один раз, все в слотах
10. Замеры	Счётчики по шагам, режим в `BattleBench`	Отчёт на 30000. Оптимизация только по его данным
11. Редактор	Файл `.fsm`, каталог задач, окно, отладка в Play	Автомат из файла ведёт себя как собранный кодом
12. Smart Objects	Объекты со слотами и резервированием	Два юнита не занимают одно место

Как программировать ИИ на C++

Всё лежит в пространстве имён `Ai`, как `Input` и `Trace`. Один заголовок: `#include <Engine/AI/Ai.h>`. Объявлять подсистему в игре не нужно: движок создаёт ИИ сам.

Пять понятий

Понятие	Что это	В Unreal
Контроллер `Ai::Controller`	Мозг юнита: запускает автомат, хранит память, видит и двигает	AIController
Автомат `Ai::StateMachine`	Дерево состояний, одно на тип юнита	StateTree
Задача `Ai::Task`	Что юнит делает в состоянии	StateTree Task, BT Task
Условие	Функция `bool`: можно ли войти или перейти	Condition, Decorator
Память `Ai::Blackboard`	Именованные значения конкретного юнита	Blackboard

Главное правило: **автомат один на всех, поэтому в объекте задачи нельзя хранить данные юнита.** Методы задачи объявлены `const`, и компилятор сам не даст ошибиться. Всё, что относится к конкретному юниту, лежит в памяти или в структуре `Data` задачи.

Первый юнит за четыре шага

Стражник ходит по маршруту, отдыхает, замечает игрока, гонится, бьёт, а потеряв цель, идёт туда, где видел её в последний раз.

Шаг 1. Память и команды

```
// Game/MyGame/Source/GuardAi.h
#pragma once
#include <Engine/AI/Ai.h>

namespace Teams
{
inline constexpr Ai::Team Player{1};
inline constexpr Ai::Team Guards{2};
}

namespace GuardAi
{
// Ключ — это имя и тип. Положить в него значение другого типа не получится.
inline constexpr Ai::Key<Ai::ObjectRef> Target{"Target"};   // за кем гонимся
inline constexpr Ai::Key<glm::vec3> LastSeen{"LastSeen"};   // где видели в последний раз

Ai::MachinePtr Machine();
}
```

Шаг 2. Своя задача

Движение, ожидание и поворот уже есть в движке. Писать нужно только то, что относится к правилам игры.

```
// Game/MyGame/Source/GuardAi.cpp
#include "GuardAi.h"

namespace GuardAi
{
struct AttackData { float cooldown = 0; };   // у каждого юнита своя копия

class Attack final : public Ai::Task<AttackData>
{
public:
    explicit Attack(float damage) : damage_(damage) {}   // настройка, общая для всех

    Ai::Status Tick(Ai::Context& ctx, AttackData& data, float dt) const override
    {
        data.cooldown -= dt;
        if (data.cooldown > 0) return Ai::Status::Running;

        const Ai::ObjectRef target = ctx.Memory().Get(Target);
        if (!target) return Ai::Status::Failed;

        ctx.GetWorld().ReportDamage(target, ctx.Self().OwnerRef(), damage_);
        data.cooldown = 1.0f;
        return Ai::Status::Running;
    }

private:
    float damage_;
};
}
```

Шаг 3. Автомат

```
// Продолжение GuardAi.cpp
namespace GuardAi
{
Ai::MachinePtr Machine()
{
    static const Ai::MachinePtr machine = [] {
        Ai::MachineBuilder m("Guard");

        auto& patrol = m.State("Patrol");            // родитель: сам не выполняется, выбирает ребёнка
        patrol.GoTo("Chase").OnEvent(Ai::Events::Spotted);

        auto& walk = patrol.State("Walk");           // первый ребёнок: сюда автомат войдёт при старте
        walk.Task<Ai::Tasks::FollowRoute>();
        walk.GoTo("Rest").OnSucceeded();
```

```
        auto& rest = patrol.State("Rest");
        rest.Task<Ai::Tasks::Wait>(2.0f);
        rest.GoTo("Walk").OnCompleted();

        auto& combat = m.State("Combat");
        combat.Task<Ai::Tasks::LookAt>(Target);      // работает, пока активен любой ребёнок
        combat.GoTo("Search").OnEvent(Ai::Events::Lost);

        auto& chase = combat.State("Chase");
        chase.Task<Ai::Tasks::MoveTo>(Target, 1.5f);
        chase.GoTo("Hit").OnSucceeded();

        auto& hit = combat.State("Hit");
        hit.Task<Attack>(10.0f);
        hit.GoTo("Chase").If(Ai::Not(Ai::Conditions::Near(Target, 2.5f)));

        auto& search = m.State("Search");
        search.Task<Ai::Tasks::MoveTo>(LastSeen, 1.0f);
        search.GoTo("Chase").OnEvent(Ai::Events::Spotted);
        search.GoTo("Patrol").OnCompleted();

        return m.Build();   // ошибка в имени состояния — исключение с понятным текстом
    }();
    return machine;
}
}
```

static здесь нужен, чтобы автомат строился один раз и все стражники делили его.

Шаг 4. Юнит

```
// Game/MyGame/Source/Guard.h
#pragma once
#include "GuardAi.h"
#include <Engine/Components/PropertyMacros.h>

ENGINE_COMPONENT("Стражник")
class Guard final : public AiCharacter
{
public:
    ENGINE_GENERATED_PROPERTIES(AiCharacter)

protected:
    void SetupAi(Ai::Controller& ai) override
    {
        const glm::vec3 home = GetWorldPosition();
        ai.SetRoute({home, home + glm::vec3(patrolLength, 0, 0)});

        Ai::Perception& senses = ai.GetPerception();
```

```
        senses.SetTeam(Teams::Guards);
        senses.EnableSight(sightRadius, 120.0f);
        senses.WriteTargetTo(GuardAi::Target);
        senses.WriteLastSeenTo(GuardAi::LastSeen);

        ai.Run(GuardAi::Machine());
    }

private:
    ENGINE_PROPERTY("Радиус зрения", 1, 200)
    float sightRadius = 20;

    ENGINE_PROPERTY("Длина маршрута", 1, 1000)
    float patrolLength = 10;
};
```

Чтобы стражник видел игрока, персонаж игрока сообщает о себе:

```
// Game/MyGame/Source/Hero.h
ENGINE_COMPONENT("Герой")
class Hero final : public PlayerCharacter
{
public:
    void BeginPlay() override
    {
        PlayerCharacter::BeginPlay();
        Ai::Get(*GetGame()).AddTarget(*this, Teams::Player);
    }
};
```

Соберите проект. Класс появится в палитре редактора, как любой `ENGINE_COMPONENT`, а свойства — в инспекторе. Поставьте стражника в сцену и нажмите Play. Дальше всё идёт само:

1. `AiCharacter::BeginPlay` создаёт контроллер и вызывает ваш `SetupAi`.
2. Автомат выбирает состояние: `Patrol`, затем его первого ребёнка `Walk`.
3. Зрение заметило героя: записало `Target` и послало событие `Spotted`.
4. Переход у `Patrol` срабатывает, автомат выходит из `Walk` и входит в `Combat`, затем в `Chase`.

Задачи

Метод	Когда вызывается	Что возвращает
`Enter`	При входе в состояние	`Running`, если нужно продолжать. `Succeeded` или `Failed`, если всё уже готово
`Tick`	При каждом обдумывании, пока задача работает	`Running`, `Succeeded` или `Failed`
`Exit`	При выходе из состояния, всегда, даже при прерывании	Ничего. `result` равен `Running`, если задачу прервали

Переопределять нужно только то, что используете. Все задачи состояния работают одновременно, первая завершившаяся завершает состояние.

Пример с парой `Enter` и `Exit`: ускорение на время состояния. `Exit` вернёт скорость, даже если состояние прервут.

```
struct SprintData { float normalSpeed = 0; };

class Sprint final : public Ai::Task<SprintData>
{
public:
    explicit Sprint(float factor) : factor_(factor) {}

    Ai::Status Enter(Ai::Context& ctx, SprintData& data) const override
    {
        auto* body = ctx.Owner<Character>();
        if (!body) return Ai::Status::Failed;
        data.normalSpeed = body->GetMoveSpeed();
        body->SetMoveSpeed(data.normalSpeed * factor_);
        return Ai::Status::Running;
    }

    void Exit(Ai::Context& ctx, SprintData& data, Ai::Status) const override
    {
        if (auto* body = ctx.Owner<Character>()) body->SetMoveSpeed(data.normalSpeed);
    }

private:
    float factor_;
};
```

Пример длительного действия: дойти до точки и понять, чем всё кончилось. Если данных нет, пишите `Ai::Task<>`.

```
inline constexpr Ai::Key<glm::vec3> CoverPoint{"CoverPoint"};

class RunToCover final : public Ai::Task<>
{
public:
    Ai::Status Enter(Ai::Context& ctx, Ai::NoData&) const override
    {
        ctx.Self().MoveTo(ctx.Memory().Get(CoverPoint), 1.0f);
        return Ai::Status::Running;
    }

    Ai::Status Tick(Ai::Context& ctx, Ai::NoData&, float) const override
    {
        switch (ctx.Self().GetMoveStatus())
        {
        case Ai::MoveStatus::Arrived:     return Ai::Status::Succeeded;
        case Ai::MoveStatus::Unreachable:
        case Ai::MoveStatus::Stuck:       return Ai::Status::Failed;
        default:                          return Ai::Status::Running;
```

```
        }
    }

    void Exit(Ai::Context& ctx, Ai::NoData&, Ai::Status result) const override
    {
        if (result == Ai::Status::Running) ctx.Self().StopMovement();   // прервали на ходу
    }
};
```

Условия

Условие — обычная функция или лямбда. Подходит везде, где нужно решение «да или нет».

```
inline constexpr Ai::Key<float> Health{"Health"};

static bool IsWounded(Ai::Context& ctx) { return ctx.Memory().Get(Health, 100.0f) < 30; }
```

```
// фрагмент
s.GoTo("Flee").If(IsWounded);
s.GoTo("Hit").If([](Ai::Context& ctx) { return ctx.DistanceTo(GuardAi::Target) < 2; });
s.GoTo("Chase").If(Ai::All(Ai::Conditions::HasKey(GuardAi::Target), Ai::Not(IsWounded)));
s.EnterIf(Ai::Conditions::HasKey(GuardAi::Target));   // без цели в это состояние не войти
```

`EnterIf` решает, можно ли войти в состояние. Родитель пробует детей по порядку объявления и входит в первого, кто разрешил. Так в автомате получается выбор как в Behavior Tree: «есть цель — бой, нет цели — патруль».

Переходы

Запись	Когда срабатывает
`.OnSucceeded()`	Задача состояния завершилась успешно
`.OnFailed()`	Задача завершилась неудачей
`.OnCompleted()`	Задача завершилась, неважно как
`.OnEvent(event)`	Пришло событие
`.If(condition)`	Проверяется при каждом обдумывании
`.OnEvent(event).If(condition)`	Пришло событие и выполнено условие
`.After(seconds)`	Добавка к любому из вариантов: переход случится с задержкой

Цель — имя состояния, `Ai::Target::Next` (следующий сосед) или `Ai::Target::Finish` (остановить автомат).

Порядок проверки такой же, как в StateTree:

- Переходы проверяются от листа к корню, внутри состояния — в порядке объявления. Срабатывает первый подходящий.
- Если цель имеет детей, автомат спускается к первому подходящему ребёнку. Если войти некуда, юнит остаётся в текущем состоянии.
- Если состояние завершилось и подходящего перехода нет ни у него, ни у родителей, автомат выбирает состояние заново от корня.
- За одно обдумывание происходит один переход, поэтому зациклиться внутри кадра нельзя.

Память

Поддерживаются `bool`, `int`, `float`, `glm::vec3`, `std::string` и `Ai::ObjectRef`. Память сохраняется в SaveGame сама.

```
// фрагмент
Ai::Blackboard& memory = ctx.Memory();
memory.Set(Health, 80.0f);
const float hp = memory.Get(Health, 100.0f);     // второй аргумент — значение по умолчанию
if (memory.Has(GuardAi::Target)) memory.Clear(GuardAi::Target);

// ObjectRef хранит ID, а не указатель: объект могли удалить.
if (auto* enemy = ctx.Resolve<Character>(memory.Get(GuardAi::Target)))
    enemy->GetWorldPosition();
```

События

Событие будит автомат сразу, не дожидаясь следующего обдумывания. Своё событие — это одна строка.

```
inline constexpr Ai::Event Alarm{"Alarm"};
```

```
// фрагмент
ai.SendEvent(Alarm);                                  // одному юниту
world.Broadcast(Alarm, {10, 0, 5}, 30.0f);            // всем в радиусе 30 м
```

Стандартные события: `Ai::Events::Spotted`, `Lost`, `Heard`, `Damaged`.

Подсостояния и повторное использование

Задачи родителя работают, пока активен любой его ребёнок. Это удобно для общего поведения: `LookAt` в `Combat` держит взгляд на цели и в `Chase`, и в `Hit`.

Общий кусок поведения можно описать один раз и вставлять в разные автоматы:

```
Ai::MachinePtr CombatMachine();   // общий бой

Ai::MachinePtr ArcherMachine()
{
    Ai::MachineBuilder m("Archer");
    m.State("Patrol").Task<Ai::Tasks::FollowRoute>();
    m.State("Combat").Linked(CombatMachine());
    return m.Build();
}
```

Службы

Служба работает в фоне с заданным интервалом и обновляет память. Это аналог Service из Behavior Tree и Global Task из StateTree.

```
inline constexpr Ai::Key<float> Danger{"Danger"};

class UpdateDanger final : public Ai::Service<>
{
public:
    void Tick(Ai::Context& ctx, Ai::NoData&, float) const override
    {
        ctx.Memory().Set(Danger, static_cast<float>(ctx.Self().GetPerception().Known().size()));
    }
};
```

```
// фрагмент
m.Service<UpdateDanger>(0.5f);     // всё время работы автомата
s.Service<UpdateDanger>(0.5f);     // только пока активно состояние
```

Движение

Из автомата проще всего использовать `Ai::Tasks::MoveTo`. Из своего кода:

```
// фрагмент
ai.MoveTo(glm::vec3(10, 0, 5));                      // в точку
ai.MoveTo(ctx.Memory().Get(GuardAi::Target), 2.0f);  // за объектом: путь обновляется сам
ai.SetFocus(ctx.Memory().Get(GuardAi::Target));      // смотреть на цель на ходу
ai.StopMovement();
```

`GetMoveStatus()`	**Значение**
`Idle`	Приказа нет
`Waiting`	Путь ещё считается
`Moving`	Идёт
`Arrived`	Дошёл
`Unreachable`	Цель недостижима, идёт к ближайшей достижимой точке
`Stuck`	Нет продвижения, новые попытки не помогли
`Unavailable`	Владелец не умеет ходить: турель, скрипт

Сетку навигации задают компоненты сцены `NavVolume` с режимом «границы», «препятствие» или «стоимость», как NavMeshBoundsVolume и NavModifierVolume в Unreal. То же можно сделать кодом. `BeginPlay` идёт уже после подготовки ИИ, поэтому сетка там доступна:

```
// фрагмент
Ai::NavGrid& nav = Ai::Get(game).Navigation();
nav.SetBounds({-100, -100}, {100, 100}, 1.0f);       // XZ, размер клетки 1 м
nav.BlockBox({-5, -1}, {5, 1});                      // стена
nav.SetCost({20, 20}, {40, 40}, 5);                  // болото: проходимо, но дорого
```

Восприятие

Зрение, слух и урон работают как AI Perception. Настройка — в `SetupAi`.

```
// фрагмент
Ai::Perception& senses = ai.GetPerception();
senses.SetTeam(Teams::Guards);
senses.EnableSight(25.0f, 120.0f, 5.0f);   // радиус, угол обзора, сколько помнит
senses.EnableHearing(40.0f);
senses.WriteTargetTo(GuardAi::Target);     // лучшая цель сама попадает в память

world.ReportNoise({3, 0, 7}, 1.0f);        // выстрел, шаги, взрыв
world.SetAttitude(Teams::Guards, Teams::Player, Ai::Attitude::Neutral);   // перемирие
```

По умолчанию разные команды враждебны, команда 0 нейтральна. Восприятие посылает события `Spotted`, `Lost`, `Heard`, `Damaged`, а список известных целей лежит в `Known()` и `Best()`.

Поиск лучшей точки

`Ai::Query` — это аналог EQS. Он создаёт точки-кандидаты, отсеивает негодные, остальным ставит оценку и отдаёт лучшую.

```
static Ai::QueryPtr CoverQuery()
{
    Ai::QueryBuilder q("Cover");
    q.Points(Ai::Generators::Ring(6.0f, 18.0f, 24));              // кандидаты вокруг юнита
    q.Filter(Ai::Tests::Reachable());                             // можно дойти
    q.Filter(Ai::Tests::HiddenFrom(GuardAi::Target));             // враг не видит
    q.Score(Ai::Tests::Distance(GuardAi::Target), Ai::Prefer::Far);
    return q.Build();
}
```

```
// фрагмент
auto& hide = s.State("Hide");
hide.Task<Ai::Tasks::RunQuery>(CoverQuery(), CoverPoint);
hide.GoTo("RunToCover").OnSucceeded();
```

Не только юниты

Автомат не привязан к персонажу. Им можно управлять чем угодно.

Турель — обычный компонент без `Character`. Она видит и думает, но не ходит:

```
#include <Engine/Components/SpecificComponents/CubeComponent.h>

Ai::MachinePtr TurretMachine();

ENGINE_COMPONENT("Турель")
class Turret final : public CubeComponent
{
public:
    void BeginPlay() override
```

```
    {
        CubeComponent::BeginPlay();
        Ai::Controller& ai = Ai::Get(*GetGame()).Spawn(*this);
        ai.GetPerception().EnableSight(30.0f, 360.0f);
        ai.Run(TurretMachine());
    }
};
```

Логика вообще без объекта сцены: сценарий боя, волны врагов, фазы босса.

```
Ai::MachinePtr BattleScript();
```

```
// фрагмент
world.SpawnDirector().Run(BattleScript());
```

Свой контроллер

Обычно хватает `SetupAi`. Если нужны свои реакции и поля, наследуйтесь от контроллера. Это тот же приём, что `Game::CreateCharacterController`.

```
class GuardController final : public Ai::Controller
{
public:
    using Ai::Controller::Controller;

    void OnPerception(const Ai::Stimulus& stimulus) override
    {
        if (stimulus.kind == Ai::Stimulus::Kind::Damage) ++woundsTaken_;
    }

    void SerializeSaveGame(SaveArchive& archive) override
    {
        Ai::Controller::SerializeSaveGame(archive);
        archive.Field("woundsTaken", woundsTaken_);
    }

private:
    int woundsTaken_ = 0;
};

class EliteGuard final : public AiCharacter
{
protected:
    std::unique_ptr<Ai::Controller> CreateController(Ai::World& world) override
    {
        return std::make_unique<GuardController>(world);
    }
};
```

Отряды и толпа

Стратегический слой — это те же автоматы, задачи и память. Отряд `Ai::Group` является контроллером, поэтому `Ai::Tasks::MoveTo` работает и для него, только вместо A* используется поле направлений. Автомат один на отряд, бойцы только двигаются.

```
Ai::MachinePtr RegimentMachine();
```

```
// фрагмент
Ai::Group& regiment = world.CreateGroup();
for (size_t i = 0; i < 100; ++i)
    regiment.AddInstance(batchId, i, 1.8f, 6.0f);    // экземпляр MeshInstance
regiment.Run(RegimentMachine());
regiment.SetFacing(0.0f);
regiment.MoveTo(glm::vec3(200, 0, 50), 5.0f);        // приказ игрока
```

Сохранение

Обычно ничего писать не нужно. Движок сохраняет активные состояния (по именам), память, приказы и восприятие.

- `Data` задач не сохраняется. После загрузки автомат возвращается в те же состояния и заново вызывает `Enter`, поэтому `MoveTo` запросит путь снова, а `Wait` начнёт отсчёт заново.
- Свои поля контроллера дописывайте в `SerializeSaveGame`.
- Ссылки на объекты храните в `Ai::ObjectRef`: при загрузке их перевяжет движок.

Отладка и тесты

Текущее состояние и журнал переходов можно получить из кода:

```
// фрагмент
world.Debug().SetVisible(true);        // над юнитами: состояние, путь, конус зрения
world.Debug().Select(ai);              // подробности: память, задачи, журнал
const std::string state = ai.ActiveState();          // "Combat/Chase"
const std::vector<std::string>& log = ai.History();  // последние переходы и причины
```

Журнал выглядит так:

```
12.40  Patrol/Walk -> Combat/Chase   событие Ai.Spotted
15.10  Combat/Chase -> Combat/Hit    задача завершилась успешно
```

ИИ работает без окна и GPU, поэтому поведение проверяется обычным тестом:

```
#include <stdexcept>

inline void GuardStartsOnPatrol()
{
    Game game;
    std::unique_ptr<GameComponent> unit = std::make_unique<Guard>();
    const AddComponentResult added = game.GetScene().AddComponent("Guard", unit);

    game.InitializeGameplay();
```

```
    for (int frame = 0; frame < 10; ++frame) game.StepProjectSimulation(0.1f);

    const Ai::Controller* ai = Ai::Get(game).Find(added.id);
    if (!ai || ai->ActiveState() != "Patrol/Walk") throw std::runtime_error("Guard is not patrolling");
}
```

Частые ошибки

Ошибка	Что будет	Как правильно
Хранить поле юнита в объекте задачи	Не скомпилируется: методы `const`	Положить в `Data` или в память
Хранить `Character*`	Падение после удаления объекта	`Ai::ObjectRef` и `ctx.Resolve`
Вызывать `Ai::Get` на каждого юнита каждый кадр	Лишний перебор подсистем	Внутри задач есть `ctx.GetWorld()`
Строить автомат для каждого юнита	Лишняя память и время	`static` внутри функции, как в примере
Считать, что `dt` — настоящие секунды	Таймеры спешат в сценах с масштабом времени	Проектам масштаб 1.0 ставит движок, в своём `Game` ставьте `SetSimulationTimeScale(1.0f)`
Менять состояние прямо из задачи	Такого API нет намеренно	Вернуть статус или послать событие, переход решает автомат
Двигать юнита сам, пока идёт `MoveTo`	Юниты дёргаются	Сначала `StopMovement`
Ждать `Spotted` без цели	Никого не видно	`AddTarget` у того, кого должны видеть

Автомат из файла и редактор

На этапе редактора автомат можно будет собрать мышью и хранить в файле, а задачи и условия останутся классами C++. Игровой код почти не меняется:

```
// эскиз этапа редактора
ai.Run(Ai::LoadMachine(*GetGame(), "/Game/AI/Guard.fsm"));

ENGINE_AI_TASK("Атаковать")
class Attack final : public Ai::Task<AttackData>
{
public:
    ENGINE_PROPERTY("Урон", 0, 1000)
    float damage = 10;
    // ...
};
```

`MachineBuilder` и файл `.fsm` собирают один и тот же `Ai::StateMachine`, поэтому оба способа можно смешивать.

Справочник

Стандартная задача	Что делает
`Ai::Tasks::Wait(seconds)`	Ждёт
`Ai::Tasks::WaitRandom(min, max)`	Ждёт случайное время
`Ai::Tasks::MoveTo(key, radius)`	Идёт к точке или объекту из памяти
`Ai::Tasks::FollowRoute()`	Идёт к следующей точке маршрута из `SetRoute`
`Ai::Tasks::LookAt(key)`	Смотрит на объект, пока активно состояние
`Ai::Tasks::SendEvent(event)`	Посылает событие себе
`Ai::Tasks::ClearKey(key)`	Очищает значение памяти
`Ai::Tasks::RunQuery(query, key)`	Ищет лучшую точку и пишет её в память
`Ai::Tasks::Log(text)`	Пишет в журнал

Стандартное условие	Истинно, если
`Ai::Conditions::HasKey(key)`	Значение есть в памяти
`Ai::Conditions::Near(key, distance)`	Цель ближе расстояния
`Ai::Conditions::CanSee(key)`	Цель видна прямо сейчас
`Ai::Conditions::TimeInState(seconds)`	Юнит в состоянии дольше этого времени
`Ai::Conditions::Chance(probability)`	Случай: 0.3 означает 30%
`Ai::Not(c)`, `Ai::All(a, b)`, `Ai::Any(a, b)`	Отрицание, «и», «или»

Решения и источники

Вопросы

1. **Нужен ли редактор автоматов мышью и когда?** Рекомендую сначала закончить C++, этапы 1–4, потом решать. Если редактор важнее толпы, он встанет перед этапом 8.
2. **Базовый класс — это `Character`?** В сообщении коммита `55d3b0c` есть «simple unit class», но в его изменениях такого класса нет. Если он есть локально, поменяется только `AiCharacter`.
3. **Как ходит толпа?** Рекомендую потоком с построением на месте. Марш в строю как в Total War можно добавить следующим слоем поверх отрядов.
4. **30000 — цель для ИИ или только для отрисовки?** От этого зависят размер клетки и срочность этапа 10.
5. **Нужны ли рельеф и этажи?** Рельеф добавит высоту клетки. Этажи и мосты сетка не потянет, тогда навигацию придётся менять на navmesh, но игровой код это не затронет.

Источники по Unreal

- https://dev.epicgames.com/documentation/en-us/unreal-engine/overview-of-state-tree-in-unreal-engine
- https://dev.epicgames.com/documentation/unreal-engine/ai-controllers-in-unreal-engine
- https://dev.epicgames.com/documentation/en-us/unreal-engine/ai-perception-in-unreal-engine
- https://dev.epicgames.com/documentation/unreal-engine/environment-query-system-overview-in-unreal-engine
- https://dev.epicgames.com/documentation/unreal-engine/smart-objects-in-unreal-engine---quick-start
- https://dev.epicgames.com/documentation/en-us/unreal-engine/ai-debugging-in-unreal-engine